

1999

Hypothesis

One can create a computer program that simulates adaptive radiation and use this program to test whether sympatric speciation (speciation within a breeding population) can occur.

Purpose

To create a computer program that simulates adaptive radiation and to use that program to test the feasibility of sympatric speciation.

Procedures

The procedures of this project fit into three categories. Each category will be addressed separately. The categories are as follows:

- A. Planning and preliminary testing
- B. Program design and construction.
- C. Program testing and experimentation

Planning and preliminary testing

After the basic idea of the project had been formulated, research was done on evolution and computer simulation. After the basic research was completed, a series of test programs were constructed in a programming language known as PERL. PERL is a unique language in many ways. First, it is a very concise language, and it allows one to achieve a

goal with fewer lines of code than many other programming languages. PERL, like C, allows regular expressions, which make pattern matching in complex strings of text very simple. One weakness of PERL that would later cause it to be abandoned as this project's programming language is its text-based orientation. PERL is a very powerful language, but it doesn't do graphics. All data would have to be represented with text. Even with this weakness, PERL was at first considered to be the best choice of a programming language for this project.

The first test programs that were constructed were basic random number generators. All these programs did was generate random numbers within a given range. These programs were constructed because random mutations are the raw material of evolution. Random number generation is essential to any program that attempts to simulate evolution. After the programs were written to generate random numbers, the project commenced into its next stage.

Program design and construction

Once basic planning was completed, the next step was to design the structure of the evolution simulator. The simulator needs to be able to loop continuously for many generations for evolution in the simulated birds to occur, so the main body of the program is constructed as a loop. In programming terms, a loop is a series of commands that are executed repetitively until a designated event occurs to terminate the cycle. The loop forms the outer rim of the program, with most of the code contained inside the loop. When the program is run, every time the code loops it constitutes a single year.

The first chunk of code inside the loop replenishes the food supply for the current year. In this project, foodstuffs (there are seven food types that will be discussed later) do not evolve or have any calculated growth rate. Every cycle (year), they reset to a given value. In a real ecosystem, the food supply may have population fluctuations that are not caused by

the birds. Removing this factor is a simplification that allows for a more controllable simulation.

The second area in the loop deals with bird ages. Every time a year passes, every living bird is aged one year. Bird life span, like many other variables, can be changed easily to allow for experimentation. When a bird reaches its maximum life span, it is removed from the system and considered to have died of old age.

The third code chunk in the loop is one of the most important. Every year, the birds in the simulation need to eat a given amount of food or they starve. Because there are seven food types, the food eaten by any bird is usually of more than one type. A bird's beak shape dictates what it can eat (this will be described later in greater detail). The order in which birds eat is determined randomly. Random numbers are also used to choose which available food types are eaten by a bird that can eat multiple food types. After all the available food is used up, any remaining birds starve. All starved birds are removed from the system.

The final step in the loop is mating. As well as tracking beak shape, bird age, life span, and other variables, the system also tracks each bird's sex. For mating, the birds are paired off until there are no more available pairs or until all remaining birds are of the same sex. (By the end of this project two non-random mating schemes were developed and experimented with. These will be discussed later.) Once the birds are paired, each pair has a random number of offspring. Each offspring is given a beak shape that is an average of its parents' beak shapes. Some offspring may also have a small mutation applied to their beak so that it is not a perfect average of their parents'. This relatively rare mutation is the key to the evolution process. Mutations are random, but the rate of mutation is a variable that can be adjusted experimentally. Each offspring is also given a random sex and a current age of zero.

The program displays statistics for each year as it passed. At the end of each year the system is populated with many new offspring and some older parents that have survived

from previous years. There is no need for a limit on the number of birds allowed, as the food supply automatically limits the number of birds in the system.

After program design was completed, the described program was written in PERL. The program was tested, and it did work as planned. Over time, offspring evolved that could eat foods that their parents couldn't. This gave them access to a large new food source, and the population of birds with the new adaptation quickly expanded. At this point, the program was known to work to some extent, but without graphs and displays it was very hard to tell exactly what was happening. After careful consideration, it was decided to abandon the PERL program and to re-write the program from scratch in a new language: RealBASIC. RealBASIC may sound like a basic language, but it is really quite powerful and effective. RealBASIC is object oriented, which means it was possible to call each bird an object and assign to those objects properties such as age and beak shape. In RealBASIC it was possible to create an array of bird objects, whereas in PERL it was necessary to create many separate arrays that were accessed together. The RealBASIC program is therefore easier to understand and is faster. RealBASIC is also a newer language. Though it uses the older BASIC command structure, it has many advanced features like multi-threading to allow more than one process to run at once. Using RealBASIC, it was not only possible to re-write the program in a much faster and more elegant format, but it was also possible to have the system draw graphs as it runs for a real-time display of what is going on. This allows the observer to identify mutations and evolutionary phenomena quickly.

An additional feature was added to the program after most programming was done. A method to let the program detect speciation was written. This method takes the morphological data from a given generation and analyzes it. The method graphically ignores the isolated mutants, leaving only the main bird clusters on the graph. It then checks to see if there is more than one cluster. If there is, then it scores it initially as speciation. As a safe guard against false positives, the program must find speciation five times in a row as the birds continue to breed. If it ever fails to find speciation, it resets and

must get five consecutive detections before it triggers. The algorithm can trigger various things when it detects speciation: it can save the array file and start a new one; it can save the file and continue running; or it can just stop running. Also included with this algorithm is an option to stop a run after a certain number of generations have passed and save the bird array to file.

The final steps in program construction were the addition of array-saving features, graphical output, options that could be changed while the program was running, and many other small enhancements to the system.

Program testing and experimentation

There are seven food types built into the system: fruit, insects, small seeds, medium seeds, large seeds, nectar from flowers with straight corollas, and nectar from flowers with curved corollas. Each food type has rules that specify what range of beak shapes can eat that food type. Every food type has different rules, but many of them overlap. For example, medium sized seeds can be eaten by birds with beaks greater than or equal to two units wide, and insects can be eaten by birds with beaks less than or equal to three units wide. This means that some birds may be able to eat more food types than others can. If a bird can eat more food types, it is more likely to survive and have offspring. This is the selective pressure that drives the evolution in the simulator. Because those offspring are mostly going to have beaks similar to their parents', most of them will also have this advantageous ability to eat more foods, so this adaptive trait should spread in the population. This is a prime example of evolution by natural selection.

The first experiment performed on the simulator tested whether evolution did indeed occur. As with all runs, the system started with two birds, one male and one female. They both start with the same beak shape. These two birds, and the beak shape rules that govern the program, are based on the Hawaiian Honeycreepers, which are known as a classic example of adaptive radiation. (The two starting birds are based loosely on the genus

Loxops in the family Drepanididae.) The starting beak shape allows these two birds to eat three of the seven food types: fruit, insects, and small seeds. The experiment was to see if, over many generations, birds evolved that could eat other food types. This did occur, and, if the mutation rate was high, it sometimes occurred very quickly. (A high mutation rate greatly accelerates the simulated evolution process but doesn't affect the outcome other than making it happen sooner.) This is an example of **adaptive radiation**, because the population starts with two identical birds and evolves into birds with different beak shapes that can eat new food types.

After it had been shown that evolution was occurring in the system, various experiments were conducted by changing different variables. The main goal of the experimentation was to try to get **sympatric speciation** (see glossary of terms for a definition) to occur. After many runs that yielded evolution but no sympatric speciation, two new mating algorithms were added. Descriptions follow.

Random mating - The random mating algorithm had been in the program since the beginning and is the most basic of the mating setups. In this algorithm, pairs of birds (a male and female) are picked completely randomly until no more pairs can be made. Any birds that can't find a mate don't have any offspring for that year but may mate the following year.

Assortative mating - In this algorithm, birds attempt to find the mate that has a beak shape most like their own. If a bird cannot find a perfect match, it picks the best available match. The only birds that do not mate are birds that can find no bird of the opposite sex.

Semi-random mating - In this algorithm, pairs of birds are picked randomly, as with the random mating algorithm. However, there is a maximum beak difference

variable that comes into play in this algorithm. If two birds are overly different (this difference is defined as the sum of the differences in the beak characteristics), then they won't be able to mate. Each bird tries to find a random mate. If a picked mate is too different, the bird tries again. If there are no suitable birds left, the bird doesn't mate.

The easiest way to detect sympatric speciation and observe beak shape changes is through the built-in scatter-plot graphs. After each year, three graphs are drawn. On each graph, every bird is represented by a dot. The three graphs show beak width vs. length, curve vs. length, and width vs. curve. By watching the dots change over time, one can tell how a group of birds or single bird is changing. By watching the graphs, one can tell when a bird dies, what a bird eats, and what a bird's beak shape is. To detect **sympatric speciation**, one needs to see two (or more) distinct clusters of dots on the graphs. If two clusters of dots are very distinct and sufficiently spaced apart, then they are considered to be separate species in this simplified model of speciation. Species in this model are identified by their morphological distinctness, and not by their reproductive isolation.

Both of the new mating algorithms were experimented with to look for sympatric speciation. The assortative mating algorithm was run many times, and didn't show any evidence of sympatric speciation. The birds in the system tended to stay together in a cluster on the graphs.

The semi-random mating algorithm, however, did produce speciation. In many runs, sympatric speciation occurred. When birds evolved that could eat one food type (large seeds), they tended to center around the large seed food type. The rest of the birds in the simulation that couldn't eat large seeds would slowly evolve to be able to eat other food types. Over time these two clusters of birds would separate even more, and, in some cases, their beaks ended up being very different looking. Because this was run under the semi-random mating algorithm, when the clusters got sufficiently different, they could not mate

with each other. This forced the species to stay as clusters. It was found that if the difference variable for the semi-random mating was set to a low number (.5), then sympatric speciation would occur more quickly.

Once some experimentation had been done, the speciation detection algorithm was utilized to automate doing hundreds of runs. Runs were done for with various difference factors, and they were compared on a graph of difference factor vs. percent of runs that speciated before 2000 generations had passed.

Results

Sympatric speciation was found to occur when some mating algorithms were used. There was no sympatric speciation with the random mating algorithm. However, some evolution always occurred with the random mating algorithm. The assortative mating algorithm also produced evolution but did not show any signs of sympatric speciation over many runs. Evolution occurred with this algorithm every time it was run. The semi-random mating algorithm always produced evolution, and, depending on the difference factor, yielded sympatric speciation most of the time. The graph shows that as the difference factor increases the number of runs that result in sympatric speciation decreases.

Conclusions

A computer program that simulates some aspects of evolution was successfully developed. The program not only simulated evolution, but also sometimes sympatric speciation. Sympatric speciation only occurred with the semi-random mating algorithm. This is logical because with that algorithm, groups of similar mutant birds tend to breed with each other. However, mutation rates in nature are much lower than in this simulation. Because the likelihood of sympatric speciation should decrease with decreased mutation rates, sympatric speciation is probably rare in nature, but it may well occur.

Sympatric speciation did not occur with the assortative mating algorithm. That is logical, as birds that can't find a mate will mate with birds less like themselves. This will tend to prevent speciation. Completely random mating is much like assortative mating in that it too tends to prevent speciation. Assortative mating could be expected to prevent divergence less than random mating, and given an extended period of time, could possibly lead to sympatric speciation, but this effect has not been observed as of yet.

This program has applications in evolutionary biology research, teaching, and computer science. In a slightly modified form, the program could be used as a teaching aid in high school or college biology courses. The program's structure and function closely resemble that of an evolutionary algorithm. Evolutionary algorithms are used in computer science to solve complex problems. In this technique, a solution is evolved much as the simulated birds evolved in this project.

Glossary of Terms*

Adaptive Radiation -- The evolutionary diversification of a taxon (adaptive type) into a number of different ecological roles or modes of life (adaptive zones, niches), usually over a relatively short period of time and leading to the appearance of a variety of forms.

Assortative Mating -- Sexual reproduction involving the non-random pairing of individuals which are more closely alike than the average (positive) or less alike than the average (negative) in respect to one or more traits.

Niche -- The ecological role of a species in a community; conceptualized as the multidimensional space, of which the coordinates are the various parameters representing the condition of existence of the species, to which it is restricted by the presence of competitor species; sometimes used loosely as an equivalent of microhabitat in the sense of the physical space occupied by a species.

Species -- A group of organisms formally recognized as distinct from other groups. A taxon of the rank of species; in the hierarchy of biological classification the category below genus; the basic unit of biological classification.

Sympatric Speciation -- The differentiation and attainment of reproductive isolation of populations that are not geographically separated and which overlap in their distributions; speciation within a breeding population.

* Adapted from Lincoln et al. (1982).

Bibliography

- Fogarty, Terence C., ed. *Evolutionary computing: AISB workshop, Brighton, U.K., April, 1996, selected papers*. Berlin: Springer, 1996.
- Futuyma, Douglas J. *Evolutionary biology, second edition*. Sunderland, MA: Sinauer Associates, Inc., 1986.
- Gibbons, Ann. On the many origins of species. *Science*, **273** (September 13, 1996), 1496-1499.
- Givinish, Thomas J. and Kenneth J. Sytsma, eds. *Molecular evolution and adaptive radiation*. Cambridge, U.K: Cambridge University Press, 1997.
- Lincoln, R.J., G.A. Boxshall, and P.F. Clark. *A dictionary of ecology, evolution, and systematics*. Cambridge, U.K.: Cambridge University Press, 1982.
- Pedrycz, Witold. *Fuzzy evolutionary computation*. Boston, MA: Kluwer Academic Publishers, 1997.
- Prata, Stephen. *Artificial life playhouse: Evolution at your fingertips*. Corte Madera, CA: Waite Group Press, 1993.